

## SLOT5:

# Populate, Match with MongoDB regex, full-fledged REST API

### Tóm tắt nội dung

13 Populate the comments with the users' information upon querying

14 Match with mongodb regex

REST API with Express, MongoDB and Mongoose:

15 Develop a full-fledged REST API server with Express, MongoDB and Mongoose

- **Populate the comments with the users' information upon querying:**

- Trong Mongoose, tính năng **populate** được sử dụng để tự động thay thế các tham chiếu (references) trong một document bằng dữ liệu thực tế từ collection khác.
- Ví dụ: Khi truy vấn comments, bạn có thể sử dụng `populate('user')` để lấy thông tin chi tiết của người dùng (như tên, email) từ collection users thay vì chỉ hiển thị ID người dùng.
- Cú pháp cơ bản:  
javascript

```
Comment.find().populate('user').exec();
```

- Điều này giúp giảm số lần truy vấn database và làm cho dữ liệu trả về đầy đủ hơn.

- **Match with MongoDB regex:**

- MongoDB hỗ trợ tìm kiếm dựa trên **regular expression** (regex) để lọc dữ liệu theo mẫu chuỗi.
- Trong Mongoose, bạn có thể sử dụng toán tử `$regex` trong truy vấn để tìm kiếm các document khớp với một biểu thức chính quy.
- Ví dụ: Tìm tất cả người dùng có tên bắt đầu bằng "A":  
javascript

```
User.find({ name: { $regex: '^A', $options: 'i' } });
```

- `$options: 'i'` khiến regex không phân biệt chữ hoa/thường. Đây là cách mạnh mẽ để tìm kiếm văn bản linh hoạt.

- **Develop a full-fledged REST API server with Express, MongoDB, and Mongoose:**

- Xây dựng một **REST API** hoàn chỉnh bao gồm các thành phần:
  - **Express:** Framework Node.js để xử lý các route, middleware, và phản hồi HTTP.
  - **MongoDB:** Cơ sở dữ liệu NoSQL để lưu trữ dữ liệu dưới dạng JSON-like documents.
  - **Mongoose:** ODM (Object Data Modeling) giúp tương tác dễ dàng giữa Node.js và MongoDB, cung cấp schema validation và các tiện ích như `populate`.
- Các bước chính:
  - Thiết lập server Express và kết nối với MongoDB qua Mongoose.
  - Định nghĩa các **schema** và **model** trong Mongoose cho các collection (ví dụ: User, Comment).
  - Xây dựng các endpoint REST (GET, POST, PUT, DELETE) để xử lý CRUD (Create, Read, Update, Delete).
  - Sử dụng middleware để xử lý lỗi, xác thực, và logging.
  - Ví dụ endpoint GET để lấy tất cả comments:  
javascript

```
app.get('/comments', async (req, res) => {
  try {
    const comments = await Comment.find().populate('user');
    res.json(comments);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

- Đảm bảo xử lý lỗi và trả về mã trạng thái HTTP phù hợp (200, 404, 500, v.v.).

## ✓ CÁC NỘI DUNG TRỌNG TÂM, CỐT LÕI CẦN LƯU Ý:

### 1. Populate the comments with the users' information upon querying

- **Khái niệm cốt lõi:** Sử dụng populate() trong Mongoose để tự động lấy dữ liệu liên quan từ collection khác (ví dụ: thông tin người dùng cho comment).
  - **Lưu ý chính:**
    - Đảm bảo schema của model (ví dụ: Comment) có tham chiếu (ref) đến model khác (ví dụ: User).
- ```
javascript
const commentSchema = new mongoose.Schema({
  text: String,
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' }
});
```
- Sử dụng populate('field') trong truy vấn để lấy dữ liệu đầy đủ.
  - Có thể populate nhiều cấp (nested populate) nếu cần, ví dụ: populate('user').populate('user.profile').
  - Xử lý lỗi khi không tìm thấy dữ liệu liên quan để tránh crash ứng dụng.
  - **Ứng dụng:** Giảm số lần truy vấn thủ công, cải thiện hiệu suất và làm dữ liệu trả về dễ đọc hơn.

### 2. Match with MongoDB regex

- **Khái niệm cốt lõi:** Sử dụng biểu thức chính quy (\$regex) để tìm kiếm linh hoạt dựa trên mẫu chuỗi trong MongoDB.
- **Lưu ý chính:**
  - Cú pháp: { field: { \$regex: 'pattern', \$options: 'i' } }, trong đó \$options: 'i' để không phân biệt hoa/thường.
  - Các mẫu regex phổ biến:
    - ^abc: Chuỗi bắt đầu bằng "abc".
    - abc\$: Chuỗi kết thúc bằng "abc".
    - .\*abc.\*: Chuỗi chứa "abc" ở bất kỳ vị trí nào.
  - Kết hợp \$regex với các toán tử khác như \$or, \$and để tìm kiếm phức tạp hơn.
  - Hiệu suất: Tránh sử dụng regex trên tập dữ liệu lớn mà không có index, vì có thể chậm.
- **Ứng dụng:** Tìm kiếm văn bản theo từ khóa, lọc dữ liệu không chính xác (fuzzy search).

### 3. Develop a full-fledged REST API server with Express, MongoDB, and Mongoose

- **Khái niệm cốt lõi:** Xây dựng một REST API hoàn chỉnh với các endpoint CRUD, sử dụng Express để xử lý HTTP, MongoDB để lưu trữ dữ liệu, và Mongoose để quản lý schema và truy vấn.
- **Lưu ý chính:**
  - **Cấu trúc project:**
    - Tách biệt logic: routes, controllers, models, và middleware.

- Ví dụ cấu trúc thư mục:  
text

```

/project
  /models
    User.js
    Comment.js
  /routes
    commentRoutes.js
  /controllers
    commentController.js
  app.js

```

- **Thiết lập kết nối:**

- Kết nối MongoDB với Mongoose:  
javascript

```

mongoose.connect('mongodb://localhost:27017/myapp', {
  useNewUrlParser: true });

```

- Kiểm tra và xử lý lỗi kết nối.

- **Định nghĩa schema và model:**

- Sử dụng Mongoose schema để định nghĩa cấu trúc dữ liệu và validation.
- Ví dụ:

javascript

```

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true }
});
const User = mongoose.model('User', userSchema);

```

- **Xây dựng endpoint REST:**

- Sử dụng các phương thức HTTP: GET (lấy dữ liệu), POST (tạo), PUT (cập nhật), DELETE (xóa).

- Ví dụ endpoint POST:

javascript

```

app.post('/comments', async (req, res) => {
  try {
    const comment = new Comment(req.body);
    await comment.save();
    res.status(201).json(comment);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

```

- **Middleware:**

- Sử dụng middleware để xử lý xác thực (authentication), phân quyền (authorization), và logging.

- Ví dụ middleware kiểm tra lỗi:

javascript

```

app.use((err, req, res, next) => {
  res.status(500).json({ message: err.message });
});

```

- **Xử lý lỗi và mã trạng thái HTTP:**

- 200: Thành công.
- 201: Tạo mới thành công.
- 400: Lỗi yêu cầu từ client.

- 404: Không tìm thấy tài nguyên.
- 500: Lỗi server.
- **Bảo mật và hiệu suất:**
  - Sử dụng `express.json()` để parse body JSON.
  - Thêm validation dữ liệu đầu vào (dùng Mongoose schema hoặc thư viện như Joi).
  - Sử dụng `async/await` để xử lý bất đồng bộ, tránh callback hell.
- **Ứng dụng:** Tạo một API hoàn chỉnh để tương tác với frontend hoặc các ứng dụng khác, đảm bảo tính mở rộng và bảo trì.

## ✓ **ỨNG DỤNG THỰC TIỄN:**

### 1. Populate the comments with the users' information upon querying

- **Ứng dụng thực tiễn:**
    - Trong các ứng dụng như **mạng xã hội** (Facebook, Twitter/X), **blog**, hoặc **diễn đàn**, khi hiển thị bình luận (comments), bạn cần hiển thị thông tin người dùng như tên, ảnh đại diện, hoặc email cùng với nội dung bình luận. Thay vì truy vấn riêng lẻ thông tin người dùng, populate giúp lấy dữ liệu liên quan một cách hiệu quả.
    - Ví dụ: Trong một ứng dụng blog, khi truy vấn danh sách bình luận của một bài viết, bạn có thể hiển thị tên và ảnh đại diện của người bình luận:
- ```
javascript
const comments = await Comment.find({ postId: postId })
  .populate('user', 'name avatar');
// Trả về comments với thông tin user (name, avatar) thay vì chỉ ObjectId
```
- **Tình huống thực tế:**
    - **Ứng dụng thương mại điện tử:** Hiển thị đánh giá sản phẩm cùng thông tin người đánh giá (tên, ngày tham gia).
    - **Ứng dụng quản lý dự án:** Hiển thị các bình luận trong một nhiệm vụ (task) kèm thông tin người tạo (tên, vai trò).
  - **Lợi ích:**
    - Giảm số lần truy vấn database, cải thiện hiệu suất.
    - Dữ liệu trả về dễ đọc, phù hợp để gửi tới frontend hoặc ứng dụng client.
    - Dễ dàng mở rộng khi cần lấy thêm thông tin từ các collection khác.

### 2. Match with MongoDB Regex

- **Ứng dụng thực tiễn:**
    - **Tìm kiếm văn bản linh hoạt** trong các ứng dụng như **công cụ tìm kiếm nội bộ**, **hệ thống quản lý nội dung (CMS)**, hoặc **ứng dụng thương mại điện tử**. Regex cho phép tìm kiếm gần đúng (fuzzy search) hoặc tìm kiếm theo mẫu cụ thể mà không cần khớp chính xác.
    - Ví dụ: Trong một ứng dụng thương mại điện tử, người dùng tìm kiếm sản phẩm với từ khóa "iphon":
- ```
javascript
const products = await Product.find({ name: { $regex: 'iphon', $options: 'i' } });
// Trả về các sản phẩm có tên chứa "iphon" (iPhone, iPhone 12, v.v.)
```
- **Tình huống thực tế:**
    - **Ứng dụng quản lý khách hàng (CRM):** Tìm kiếm khách hàng theo tên hoặc email một cách linh hoạt, ngay cả khi người dùng nhập sai chính tả (ví dụ: "Jon" thay vì "John").

- **Hệ thống quản lý tài liệu:** Tìm kiếm tài liệu dựa trên tiêu đề hoặc nội dung chứa một từ khóa cụ thể.
- **Ứng dụng chat:** Tìm kiếm tin nhắn hoặc người dùng dựa trên một phần nội dung hoặc tên.
- **Lợi ích:**
  - Cung cấp trải nghiệm tìm kiếm thân thiện với người dùng, hỗ trợ tìm kiếm không chính xác.
  - Linh hoạt khi xử lý dữ liệu không đồng nhất (ví dụ: tên viết sai chính tả).
  - Có thể kết hợp với các tính năng như autocomplete hoặc gợi ý tìm kiếm.

### 3. Develop a full-fledged REST API server with Express, MongoDB, and Mongoose

- **Ứng dụng thực tiễn:**
  - Xây dựng một **REST API** là nền tảng cho hầu hết các ứng dụng web và di động hiện đại, cung cấp giao tiếp giữa **frontend** (React, Vue, Angular) hoặc **ứng dụng di động** (iOS, Android) với backend. API này cho phép thực hiện các thao tác CRUD trên dữ liệu.
  - Ví dụ: Trong một ứng dụng đặt đồ ăn trực tuyến:
    - **GET /restaurants:** Lấy danh sách nhà hàng.
    - **POST /orders:** Tạo đơn hàng mới.
    - **PUT /orders/:id:** Cập nhật trạng thái đơn hàng.
    - **DELETE /orders/:id:** Hủy đơn hàng.

javascript

```
app.get('/restaurants', async (req, res) => {
  try {
    const restaurants = await Restaurant.find();
    res.json(restaurants);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

- **Tình huống thực tế:**
  - **Ứng dụng thương mại điện tử:** API để quản lý sản phẩm, giỏ hàng, đơn hàng, và thông tin người dùng.
  - **Ứng dụng quản lý công việc (như Trello):** API để tạo, cập nhật, xóa bảng, danh sách, và thẻ công việc.
  - **Ứng dụng mạng xã hội:** API để đăng bài, bình luận, thích bài viết, và quản lý thông tin người dùng.
- **Lợi ích:**
  - **Tính mở rộng:** REST API dễ dàng tích hợp với các nền tảng khác (web, mobile, IoT).
  - **Tái sử dụng:** Một API có thể phục vụ nhiều ứng dụng client khác nhau.
  - **Bảo trì dễ dàng:** Với cấu trúc tách biệt (routes, controllers, models), mã nguồn dễ bảo trì và mở rộng.
  - **Bảo mật:** Có thể tích hợp xác thực (JWT, OAuth) và phân quyền để bảo vệ endpoint.

### Tình huống tổng hợp trong thực tế

Giả sử bạn đang phát triển một **ứng dụng mạng xã hội**:

- **Populate:** Khi hiển thị một bài viết, bạn sử dụng populate để lấy thông tin người đăng bài và danh sách bình luận kèm thông tin người bình luận.

javascript

```
const post = await
Post.findById(postId).populate('author').populate('comments.user');
```

- **Regex:** Người dùng tìm kiếm bài viết hoặc người dùng khác dựa trên từ khóa: javascript

```
const posts = await Post.find({ content: { $regex: req.query.keyword,
$options: 'i' } });
```

- **REST API:** Cung cấp các endpoint để:
  - Tạo bài viết (POST /posts).
  - Lấy danh sách bài viết (GET /posts).
  - Thích bài viết (PUT /posts/:id/like).
  - Xóa bài viết (DELETE /posts/:id).

### Lưu ý khi áp dụng thực tế

- **Hiệu suất:** Sử dụng index cho các trường thường xuyên tìm kiếm bằng regex hoặc populate để tăng tốc độ truy vấn.
- **Bảo mật:**
  - Với REST API, luôn kiểm tra và xác thực dữ liệu đầu vào để tránh các lỗi như SQL injection (trong MongoDB là NoSQL injection).
  - Sử dụng JWT hoặc OAuth để bảo vệ các endpoint nhạy cảm.
- **Xử lý lỗi:** Đảm bảo API trả về thông báo lỗi rõ ràng và mã trạng thái HTTP phù hợp để hỗ trợ debug và trải nghiệm người dùng.
- **Tài liệu API:** Sử dụng công cụ như Swagger hoặc Postman để tạo tài liệu API, giúp nhóm phát triển frontend hoặc bên thứ ba dễ dàng tích hợp.

## CHI TIẾT MỘT SỐ NỘI DUNG:

### 1. Populate the Comments with the Users' Information upon Querying

#### Khái niệm cơ bản

- **Populate** là một tính năng của Mongoose, cho phép tự động thay thế các tham chiếu (ObjectId) trong một document bằng dữ liệu thực tế từ collection liên quan. Điều này đặc biệt hữu ích khi làm việc với các mối quan hệ (relationships) trong MongoDB, một cơ sở dữ liệu NoSQL không hỗ trợ join như SQL.
- Ví dụ: Trong một ứng dụng blog, mỗi bình luận (Comment) có một trường user chứa ObjectId tham chiếu đến tài liệu người dùng trong collection users. Khi truy vấn bình luận, bạn muốn lấy thêm thông tin người dùng (như tên, ảnh đại diện) thay vì chỉ ID.

#### Chi tiết kỹ thuật

- **Cách hoạt động:**
  - Trong schema của Mongoose, bạn định nghĩa một trường với type: `mongoose.Schema.Types.ObjectId` và `ref` để chỉ định collection liên quan.
  - Khi truy vấn, sử dụng phương thức `.populate('field')` để lấy dữ liệu từ collection được tham chiếu.

- **Cú pháp:**

javascript

```
const commentSchema = new mongoose.Schema({
  text: { type: String, required: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' }
});
```

```
const Comment = mongoose.model('Comment', commentSchema);
```

```
// Truy vấn và populate
```

```
Comment.find()
  .populate('user') // Lấy toàn bộ thông tin của user
  .populate('post', 'title') // Chỉ lấy trường title của post
  .exec();
```

- **Populate nhiều cấp (nested populate):**

- Nếu User có một trường tham chiếu khác (ví dụ: profile), bạn có thể populate sâu hơn:

javascript

```
Comment.find()
  .populate({
    path: 'user',
    populate: { path: 'profile' }
  })
  .exec();
```

- **Chọn lọc trường (select):**

- Để tối ưu hiệu suất, bạn có thể chỉ định các trường cần lấy:

javascript

```
Comment.find().populate('user', 'name email').exec();
```

#### Ứng dụng thực tế

- **Mạng xã hội:** Hiển thị bình luận cùng tên và ảnh đại diện của người dùng. Ví dụ, khi tải danh sách bình luận dưới một bài đăng trên Twitter/X, bạn cần tên và avatar của người bình luận.
- **Thương mại điện tử:** Hiển thị đánh giá sản phẩm kèm thông tin người đánh giá (tên, ngày tham gia).

- **Ứng dụng quản lý dự án:** Hiển thị các bình luận trong một task kèm thông tin người tạo (tên, vai trò).
- **Ví dụ thực tế:**
  - Giả sử bạn xây dựng một ứng dụng blog. Khi người dùng xem một bài viết, bạn muốn hiển thị danh sách bình luận với tên và ảnh đại diện của người bình luận: javascript

```
const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  comments: [{ type: mongoose.Schema.Types.ObjectId, ref:
'Comment' }]
});

const userSchema = new mongoose.Schema({
  name: String,
  avatar: String
});

const commentSchema = new mongoose.Schema({
  text: String,
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' }
});

const Post = mongoose.model('Post', postSchema);
const User = mongoose.model('User', userSchema);
const Comment = mongoose.model('Comment', commentSchema);

// Truy vấn bài viết với bình luận và thông tin người dùng
async function getPostWithComments(postId) {
  try {
    const post = await Post.findById(postId)
      .populate({
        path: 'comments',
        populate: { path: 'user', select: 'name avatar' }
      });
    return post;
  } catch (err) {
    throw new Error(err.message);
  }
}
```

### Lưu ý quan trọng

- **Hiệu suất:** Populate thực hiện các truy vấn bổ sung để lấy dữ liệu từ collection khác. Nếu bạn populate quá nhiều trường hoặc trên tập dữ liệu lớn, hiệu suất có thể bị ảnh hưởng. Hãy chọn lọc trường bằng select.
- **Xử lý lỗi:** Nếu tài liệu được tham chiếu không tồn tại (ví dụ: user đã bị xóa), trường populate sẽ trả về null. Đảm bảo kiểm tra điều này trong code frontend hoặc backend.
- **Tối ưu hóa:** Sử dụng index trên các trường tham chiếu (ObjectId) để tăng tốc độ truy vấn.

### Mẹo thực hành

- Thử populate với một ứng dụng nhỏ: Tạo một API trả về danh sách bình luận với thông tin người dùng.
- Kiểm tra hiệu suất bằng cách so sánh thời gian truy vấn với và không sử dụng populate.
- Thử populate nhiều cấp (ví dụ: bình luận -> người dùng -> thông tin hồ sơ) để hiểu cách hoạt động với dữ liệu phức tạp.



## 2. Match with MongoDB Regex

### Khái niệm cơ bản

- MongoDB hỗ trợ tìm kiếm dựa trên **biểu thức chính quy (regex)**, cho phép lọc dữ liệu theo mẫu chuỗi. Điều này hữu ích khi bạn cần tìm kiếm gần đúng (fuzzy search) hoặc tìm kiếm theo một mẫu cụ thể.
- Trong Mongoose, bạn sử dụng toán tử \$regex trong truy vấn để áp dụng regex.

### Chi tiết kỹ thuật

- **Cú pháp cơ bản:**  
javascript

```
// Tìm tất cả người dùng có tên bắt đầu bằng "A"
User.find({ name: { $regex: '^A', $options: 'i' } });
```

- ^A: Khớp với chuỗi bắt đầu bằng "A".
- \$options: 'i': Không phân biệt chữ hoa/thường (case-insensitive).

- **Các mẫu regex phổ biến:**
  - ^abc: Chuỗi bắt đầu bằng "abc".
  - abc\$: Chuỗi kết thúc bằng "abc".
  - .\*abc.\*: Chuỗi chứa "abc" ở bất kỳ vị trí nào.
  - [a-z]: Khớp với bất kỳ ký tự nào từ a đến z.

- **Kết hợp với các toán tử khác:**  
javascript

```
// Tìm người dùng có tên chứa "john" hoặc "jane"
User.find({
  $or: [
    { name: { $regex: 'john', $options: 'i' } },
    { name: { $regex: 'jane', $options: 'i' } }
  ]
});
```

- **Sử dụng với biến động:**
  - Khi người dùng nhập từ khóa từ giao diện, bạn có thể truyền trực tiếp vào \$regex:  
javascript

```
const keyword = req.query.search; // Ví dụ: "iphon"
Product.find({ name: { $regex: keyword, $options: 'i' } });
```

### Ứng dụng thực tế

- **Công cụ tìm kiếm:** Trong các ứng dụng như thương mại điện tử, người dùng tìm kiếm sản phẩm với từ khóa không chính xác (ví dụ: "iphon" thay vì "iPhone"). Regex giúp trả về kết quả phù hợp.
- **Quản lý khách hàng (CRM):** Tìm kiếm khách hàng dựa trên tên, email, hoặc số điện thoại với các mẫu không đầy đủ.
- **Ứng dụng quản lý tài liệu:** Tìm kiếm tài liệu dựa trên tiêu đề hoặc nội dung chứa từ khóa cụ thể.
- **Ví dụ thực tế:**
  - Trong một ứng dụng thương mại điện tử, bạn muốn tìm kiếm sản phẩm dựa trên từ khóa người dùng nhập:  
javascript

```
const productSchema = new mongoose.Schema({
  name: String,
  price: Number,
  category: String
});

const Product = mongoose.model('Product', productSchema);
```

```

async function searchProducts(keyword) {
  try {
    const products = await Product.find({
      name: { $regex: keyword, $options: 'i' }
    });
    return products;
  } catch (err) {
    throw new Error(err.message);
  }
}

// Gọi hàm
searchProducts('iphon'); // Trả về iPhone, iPhone 12, iPhone 13,
v.v.

```

### Lưu ý quan trọng

- **Hiệu suất:** Regex trên tập dữ liệu lớn có thể chậm nếu không có index. Sử dụng text index cho tìm kiếm văn bản:

```

productSchema.index({ name: 'text' });
Product.find({ $text: { $search: keyword } });

```

- **Bảo mật:** Nếu từ khóa do người dùng nhập, hãy sanitize input để tránh các cuộc tấn công regex injection (dù hiếm trong MongoDB).
- **Hạn chế:** Regex không phù hợp với tìm kiếm ngôn ngữ tự nhiên phức tạp. Trong trường hợp này, cân nhắc sử dụng công cụ như Elasticsearch.

### Mẹo thực hành

- Thử xây dựng một API tìm kiếm sản phẩm với từ khóa người dùng nhập.
- Kết hợp regex với các bộ lọc khác (ví dụ: tìm sản phẩm chứa từ khóa "phone" trong danh mục "electronics").
- So sánh hiệu suất giữa \$regex và text search của MongoDB trên tập dữ liệu lớn.

## 3. Develop a Full-Fledged REST API Server with Express, MongoDB, and Mongoose

### Khái niệm cơ bản

- Một **REST API** (Representational State Transfer) là một giao diện lập trình ứng dụng tuân theo các nguyên tắc REST, cho phép các ứng dụng giao tiếp với nhau qua HTTP, sử dụng các phương thức như GET, POST, PUT, DELETE để thực hiện các thao tác CRUD.
- **Công cụ sử dụng:**
  - **Express:** Framework Node.js để xử lý các route HTTP, middleware, và phản hồi.
  - **MongoDB:** Cơ sở dữ liệu NoSQL lưu trữ dữ liệu dưới dạng JSON-like documents.
  - **Mongoose:** ODM (Object Data Modeling) để định nghĩa schema, validate dữ liệu, và tương tác với MongoDB.

### Chi tiết kỹ thuật

- **Cấu trúc project:**
  - Tách biệt code thành các module để dễ bảo trì:

```

/project
  /models
    User.js
    Comment.js
    Post.js
  /routes

```

```

    userRoutes.js
    commentRoutes.js
    postRoutes.js
  /controllers
    userController.js
    commentController.js
    postController.js
  /middleware
    auth.js
    errorHandler.js
  app.js

```

- **Kết nối MongoDB:**

javascript

```

const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp', {
  useNewUrlParser: true,
  useUnifiedTopology: true
})
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.error('Connection error:', err));

```

- **Định nghĩa schema và model:**

javascript

```

const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true }
});

const postSchema = new mongoose.Schema({
  title: { type: String, required: true },
  content: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  comments: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Comment' }]
});

const commentSchema = new mongoose.Schema({
  text: { type: String, required: true },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' }
});

const User = mongoose.model('User', userSchema);
const Post = mongoose.model('Post', postSchema);
const Comment = mongoose.model('Comment', commentSchema);

```

- **Xây dựng các endpoint REST:**

- **GET /posts:** Lấy danh sách bài viết.

javascript

```

app.get('/posts', async (req, res) => {
  try {
    const posts = await Post.find().populate('author',
'name').populate('comments');
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

- **POST /posts:** Tạo bài viết mới.

javascript

```
app.post('/posts', async (req, res) => {
  try {
    const post = new Post({
      title: req.body.title,
      content: req.body.content,
      author: req.body.authorId
    });
    await post.save();
    res.status(201).json(post);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

- **PUT /posts/:id:** Cập nhật bài viết.

javascript

```
app.put('/posts/:id', async (req, res) => {
  try {
    const post = await Post.findByIdAndUpdate(
      req.params.id,
      { title: req.body.title, content: req.body.content },
      { new: true }
    );
    if (!post) return res.status(404).json({ message: 'Post not found' });
    res.json(post);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});
```

- **DELETE /posts/:id:** Xóa bài viết.

javascript

```
app.delete('/posts/:id', async (req, res) => {
  try {
    const post = await Post.findByIdAndDelete(req.params.id);
    if (!post) return res.status(404).json({ message: 'Post not found' });
    res.json({ message: 'Post deleted' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

- **Middleware:**

- **Xác thực (authentication):**

javascript

```
const jwt = require('jsonwebtoken');

function authMiddleware(req, res, next) {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ message: 'No token provided' });
  try {
    const decoded = jwt.verify(token, 'secret_key');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid token' });
  }
}
```

```

    res.status(401).json({ message: 'Invalid token' });
  }
}

// Áp dụng middleware cho endpoint
app.post('/posts', authMiddleware, async (req, res) => { ... });

```

#### ○ Xử lý lỗi:

javascript

```

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Something went wrong!' });
});

```

### Ứng dụng thực tế

- **Thương mại điện tử:** API để quản lý sản phẩm, giỏ hàng, đơn hàng, và đánh giá.
- **Mạng xã hội:** API để đăng bài, bình luận, thích bài viết, và quản lý hồ sơ người dùng.
- **Ứng dụng quản lý công việc:** API để tạo, cập nhật, xóa nhiệm vụ, bảng, và danh sách.
- **Ví dụ thực tế:**

- Xây dựng một API cho ứng dụng blog:

- **GET /posts:** Lấy tất cả bài viết với thông tin tác giả và bình luận.
- **POST /comments:** Tạo bình luận mới cho bài viết.
- **GET /search:** Tìm kiếm bài viết theo từ khóa sử dụng regex.

javascript

```

app.get('/search', async (req, res) => {
  try {
    const keyword = req.query.q;
    const posts = await Post.find({
      $or: [
        { title: { $regex: keyword, $options: 'i' } },
        { content: { $regex: keyword, $options: 'i' } }
      ]
    }).populate('author', 'name');
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

### Lưu ý quan trọng

- **Bảo mật:**
  - Validate dữ liệu đầu vào bằng Mongoose schema hoặc thư viện như Joi.
  - Sử dụng HTTPS để bảo vệ dữ liệu truyền tải.
  - Áp dụng rate limiting để ngăn chặn tấn công DDoS.
- **Hiệu suất:**
  - Sử dụng index trên các trường thường xuyên truy vấn.
  - Tối ưu hóa populate bằng cách chọn lọc trường.
  - Sử dụng phân trang (pagination) cho các endpoint trả về danh sách lớn:

javascript

```

app.get('/posts', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const skip = (page - 1) * limit;
  try {
    const posts = await
    Post.find().skip(skip).limit(limit).populate('author');
    res.json(posts);
  } catch (err) {

```

```

        res.status(500).json({ message: err.message });
    }
});

```

- **Tài liệu API:** Sử dụng Swagger hoặc Postman để tạo tài liệu API rõ ràng.

### Mẹo thực hành

- Xây dựng một API nhỏ với các endpoint CRUD cho một ứng dụng đơn giản (ví dụ: blog hoặc todo list).
- Tích hợp xác thực JWT để bảo vệ các endpoint.
- Thử kết hợp populate và regex trong một endpoint tìm kiếm (ví dụ: tìm kiếm bài viết và hiển thị thông tin tác giả).
- Kiểm tra hiệu suất API với công cụ như Postman hoặc JMeter.

## Tình huống tổng hợp thực tế

Giả sử bạn phát triển một **ứng dụng mạng xã hội**:

1. **Populate:** Khi hiển thị một bài viết, bạn sử dụng populate để lấy thông tin tác giả và bình luận:

javascript

CollapseWrapRun

```

const post = await Post.findById(postId)
  .populate('author', 'name avatar')
  .populate({
    path: 'comments',
    populate: { path: 'user', select: 'name avatar' }
  });

```

2. **Regex:** Cho phép người dùng tìm kiếm bài viết hoặc người dùng theo từ khóa:

javascript

```

app.get('/search', async (req, res) => {
  try {
    const keyword = req.query.q;
    const posts = await Post.find({
      $or: [
        { title: { $regex: keyword, $options: 'i' } },
        { content: { $regex: keyword, $options: 'i' } }
      ]
    }).populate('author', 'name');
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

3. **REST API:** Cung cấp các endpoint:

- **POST /posts:** Tạo bài viết mới (yêu cầu xác thực).
- **GET /posts/:id:** Lấy chi tiết bài viết với thông tin tác giả và bình luận.
- **POST /comments:** Tạo bình luận mới cho bài viết.
- **DELETE /posts/:id:** Xóa bài viết (chỉ tác giả hoặc admin được phép).

## Kết luận

- **Populate** giúp bạn làm việc hiệu quả với các mối quan hệ trong MongoDB, giảm số lần truy vấn và cung cấp dữ liệu đầy đủ cho frontend.
- **Regex** cung cấp khả năng tìm kiếm linh hoạt, đặc biệt trong các ứng dụng cần tìm kiếm văn bản hoặc lọc dữ liệu không chính xác.
- **REST API với Express, MongoDB, Mongoose** là nền tảng để xây dựng các ứng dụng web và di động hiện đại, với các endpoint CRUD, bảo mật, và hiệu suất tối ưu.

## KINH NGHIỆM THỰC TIỄN, VẤN ĐỀ CẦN LƯU Ý, VÀ CÁC MẸO (TIPS)

### 1. Populate the Comments with the Users' Information upon Querying

#### Kinh nghiệm thực tiễn

- **Ứng dụng phổ biến:** Populate rất hữu ích trong các ứng dụng có mối quan hệ giữa các collection, như mạng xã hội (bài viết, bình luận, người dùng), thương mại điện tử (sản phẩm, đánh giá, người mua), hoặc hệ thống quản lý nội dung (bài viết, danh mục, tác giả).
- **Tối ưu hóa dữ liệu trả về:** Trong thực tế, không phải lúc nào bạn cũng cần toàn bộ dữ liệu từ collection được tham chiếu. Ví dụ, khi hiển thị bình luận, bạn thường chỉ cần tên và ảnh đại diện của người dùng, không cần toàn bộ thông tin như email hay mật khẩu.
- **Kết hợp với frontend:** Populate giúp giảm số lần gọi API từ frontend, vì dữ liệu liên quan được trả về trong một lần truy vấn. Điều này cải thiện trải nghiệm người dùng (UX) bằng cách giảm thời gian tải.

#### Vấn đề cần lưu ý

##### 1. Hiệu suất:

- Populate thực hiện các truy vấn bổ sung tới database, có thể làm chậm ứng dụng nếu sử dụng trên tập dữ liệu lớn hoặc populate nhiều cấp (nested populate).
- **Giải pháp:** Chỉ populate các trường cần thiết bằng cách sử dụng select. Ví dụ: javascript

```
Comment.find().populate('user', 'name avatar').exec();
```

- Nếu có nhiều cấp populate, hãy cân nhắc cấu trúc lại schema để giảm sự phụ thuộc vào các mối quan hệ phức tạp.

##### 2. Dữ liệu không tồn tại:

- Nếu tài liệu được tham chiếu (ví dụ: user) bị xóa, trường populate sẽ trả về null. Điều này có thể gây lỗi ở frontend nếu không xử lý đúng.
- **Giải pháp:** Kiểm tra dữ liệu trả về trước khi hiển thị: javascript

```
if (comment.user) {  
  // Hiển thị thông tin user  
} else {  
  // Hiển thị thông báo "Người dùng không tồn tại"  
}
```

##### 3. Thiết kế schema:

- Populate chỉ hoạt động khi bạn đã định nghĩa đúng ref trong schema. Nếu sai ref (ví dụ: tham chiếu đến một collection không tồn tại), populate sẽ thất bại mà không báo lỗi rõ ràng.
- **Giải pháp:** Kiểm tra kỹ schema trước khi sử dụng populate.

#### Mẹo (Tips)

##### 1. Chọn lọc trường:

- Luôn sử dụng select để chỉ lấy các trường cần thiết, giảm tải cho database: javascript

```
Comment.find().populate('user', 'name avatar -_id').exec(); // -  
_id loại bỏ trường _id
```

##### 2. Populate nhiều cấp có kiểm soát:

- Khi cần populate nhiều cấp, hãy chia nhỏ truy vấn nếu có thể để dễ debug và tối ưu: javascript

```
const comments = await Comment.find().populate({  
  path: 'user',  
  select: 'name',  
});
```

```
    populate: { path: 'profile', select: 'bio' }
  });
```

### 3. Tối ưu hóa với index:

- Đảm bảo các trường tham chiếu (ObjectId) có index để tăng tốc độ truy vấn:

```
commentSchema.index({ user: 1 });
```

### 4. Kiểm tra dữ liệu trước khi populate:

- Trước khi populate, kiểm tra xem collection có dữ liệu hay không để tránh truy vấn không cần thiết:

```
const commentCount = await Comment.countDocuments();
if (commentCount === 0) return res.json([]);
```

### 5. Sử dụng lean() để tăng tốc:

- Nếu bạn chỉ cần dữ liệu thô và không cần các phương thức Mongoose, hãy sử dụng .lean() để trả về JSON đơn giản, giảm thời gian xử lý:

```
Comment.find().populate('user', 'name').lean().exec();
```

## 2. Match with MongoDB Regex

### Kinh nghiệm thực tiễn

- Tìm kiếm linh hoạt:** Regex rất hữu ích trong các ứng dụng cần tìm kiếm văn bản, như tìm kiếm sản phẩm, bài viết, hoặc thông tin khách hàng. Trong thực tế, người dùng thường nhập từ khóa không chính xác (ví dụ: "iphon" thay vì "iPhone"), và regex giúp xử lý các trường hợp này.
- Kết hợp với các tính năng khác:** Regex thường được sử dụng cùng với các bộ lọc khác (như danh mục, giá) để tạo ra các công cụ tìm kiếm mạnh mẽ.
- Ứng dụng phổ biến:**
  - Trong thương mại điện tử: Tìm kiếm sản phẩm dựa trên tên hoặc mô tả.
  - Trong CRM: Tìm kiếm khách hàng hoặc liên hệ dựa trên tên, email, hoặc số điện thoại.
  - Trong hệ thống chat: Tìm kiếm tin nhắn hoặc người dùng dựa trên từ khóa.

### Vấn đề cần lưu ý

#### 1. Hiệu suất:

- Regex trên tập dữ liệu lớn có thể rất chậm, đặc biệt nếu không có index hoặc sử dụng các mẫu regex phức tạp.
- Giải pháp:** Sử dụng **text index** của MongoDB cho tìm kiếm văn bản thay vì regex khi có thể:

```
productSchema.index({ name: 'text', description: 'text' });
Product.find({ $text: { $search: 'iphone' } });
```

- Nếu bắt buộc dùng regex, hãy sử dụng các mẫu cụ thể (ví dụ: ^abc thay vì .\*abc.\*) để giảm thời gian xử lý.

#### 2. Bảo mật:

- Từ khóa do người dùng nhập có thể chứa các ký tự đặc biệt, gây ra lỗi hoặc hiệu suất kém.
- Giải pháp:** Sanitize input trước khi truyền vào \$regex:

```
const escapeRegex = (text) => text.replace(/[-
[\]{}()*+?.,\\^$|#\s]/g, '\\$&');
```



```
const keyword = escapeRegex(req.query.search);
Product.find({ name: { $regex: keyword, $options: 'i' } }));
```

### 3. Hạn chế của regex:

- Regex không phù hợp với tìm kiếm ngôn ngữ tự nhiên phức tạp hoặc tìm kiếm theo ngữ nghĩa. Trong các trường hợp này, cần sử dụng các công cụ như Elasticsearch.
- **Giải pháp:** Đánh giá yêu cầu dự án để quyết định khi nào nên dùng regex và khi nào cần công cụ khác.

## Mẹo (Tips)

### 1. Kết hợp với các bộ lọc:

- Kết hợp regex với các toán tử MongoDB khác để tăng độ chính xác:

```
Product.find({
  $and: [
    { name: { $regex: 'iphon', $options: 'i' } },
    { category: 'electronics' }
  ]
});
```

### 2. Sử dụng text index thay thế:

- Nếu ứng dụng yêu cầu tìm kiếm văn bản thường xuyên, hãy chuyển sang text index để tăng hiệu suất:

```
Product.find({ $text: { $search: 'iphone samsung' } }).sort({
  score: { $meta: 'textScore' } });
```

### 3. Hỗ trợ autocomplete:

- Sử dụng regex để xây dựng tính năng gợi ý tìm kiếm:

```
app.get('/autocomplete', async (req, res) => {
  const keyword = req.query.q;
  const suggestions = await Product.find({
    name: { $regex: `^${keyword}`, $options: 'i' }
  }).limit(5).select('name');
  res.json(suggestions);
});
```

### 4. Kiểm tra từ khóa trước:

- Tránh chạy regex nếu từ khóa rỗng hoặc không hợp lệ:

```
if (!keyword || keyword.length < 2) {
  return res.status(400).json({ message: 'Keyword too short' });
}
```

### 5. Sử dụng biến động:

- Cho phép người dùng tùy chỉnh cách tìm kiếm (ví dụ: khớp chính xác, chứa, bắt đầu bằng):

```
const type = req.query.type; // 'exact', 'contains', 'startsWith'
let regexPattern;
if (type === 'startsWith') regexPattern = `^${keyword}`;
else if (type === 'exact') regexPattern = `^${keyword}$`;
else regexPattern = keyword;
Product.find({ name: { $regex: regexPattern, $options: 'i' } }));
```

### 3. Develop a Full-Fledged REST API Server with Express, MongoDB, and Mongoose

#### Kinh nghiệm thực tiễn

- **Tính mở rộng:** REST API là nền tảng cho hầu hết các ứng dụng web và di động hiện đại. Một API được thiết kế tốt có thể phục vụ nhiều client (web, mobile, IoT) và dễ dàng mở rộng khi dự án phát triển.
- **Tách biệt logic:** Trong thực tế, các dự án lớn thường tách biệt routes, controllers, models, và middleware để dễ bảo trì và làm việc nhóm. Ví dụ:  
text

```
/controllers/postController.js
/routes/postRoutes.js
/models/Post.js
```
- **Xác thực và phân quyền:** Hầu hết các API thực tế đều yêu cầu xác thực (authentication) và phân quyền (authorization) để bảo vệ dữ liệu. Ví dụ, chỉ người tạo bài viết mới được phép chỉnh sửa hoặc xóa bài viết.
- **Ứng dụng phổ biến:**
  - Thương mại điện tử: Quản lý sản phẩm, đơn hàng, giỏ hàng.
  - Mạng xã hội: Đăng bài, bình luận, thích bài viết.
  - Ứng dụng quản lý: Quản lý công việc, dự án, hoặc tài liệu.

#### Vấn đề cần lưu ý

##### 1. Bảo mật:

- **Xác thực:** Sử dụng JWT, OAuth, hoặc các phương pháp xác thực khác để bảo vệ endpoint:  
javascript

```
const jwt = require('jsonwebtoken');
function authMiddleware(req, res, next) {
  const token = req.header('Authorization')?.replace('Bearer ', '');
  if (!token) return res.status(401).json({ message: 'No token provided' });
  try {
    const decoded = jwt.verify(token, 'secret_key');
    req.user = decoded;
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid token' });
  }
}
```

- **NoSQL Injection:** Validate dữ liệu đầu vào để tránh các cuộc tấn công NoSQL injection. Sử dụng thư viện như Joi hoặc Mongoose validation:  
javascript

```
const Joi = require('joi');
const postSchema = Joi.object({
  title: Joi.string().min(3).required(),
  content: Joi.string().min(10).required(),
  authorId: Joi.string().hex().length(24).required()
});
```

- **HTTPS:** Đảm bảo API chạy trên HTTPS để bảo vệ dữ liệu truyền tải.

##### 2. Hiệu suất:

- **Phân trang (pagination):** Tránh trả về toàn bộ dữ liệu trong một lần gọi API. Sử dụng skip và limit:  
javascript

```
app.get('/posts', async (req, res) => {
  const page = parseInt(req.query.page) || 1;
  const limit = parseInt(req.query.limit) || 10;
  const skip = (page - 1) * limit;
  try {
    const posts = await
Post.find().skip(skip).limit(limit).populate('author');
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});
```

- **Caching:** Sử dụng Redis hoặc caching trong bộ nhớ để lưu trữ các truy vấn thường xuyên.
- **Index:** Tạo index cho các trường thường xuyên truy vấn để tăng tốc độ:

```
postSchema.index({ title: 1, createdAt: -1 });
```

### 3. Xử lý lỗi:

- Luôn trả về mã trạng thái HTTP phù hợp (200, 201, 400, 404, 500) và thông báo lỗi rõ ràng:

javascript

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Internal server error', error:
err.message });
});
```

- Xử lý các trường hợp đặc biệt như tài nguyên không tìm thấy (404) hoặc dữ liệu không hợp lệ (400).

### 4. Tài liệu API:

- Trong thực tế, các dự án lớn yêu cầu tài liệu API rõ ràng để nhóm phát triển frontend hoặc bên thứ ba sử dụng. Sử dụng Swagger hoặc Postman để tạo tài liệu tự động.

### 5. Versioning:

- Khi API phát triển, hãy sử dụng versioning (ví dụ: /api/v1/posts) để tránh phá vỡ các client hiện tại khi thêm tính năng mới.

## Mẹo (Tips)

### 1. Cấu trúc code rõ ràng:

- Tách biệt logic thành các file riêng (routes, controllers, models) để dễ bảo trì:

javascript

```
// routes/postRoutes.js
const express = require('express');
const router = express.Router();
const postController = require('../controllers/postController');

router.get('/', postController.getPosts);
router.post('/', postController.createPost);

module.exports = router;

// controllers/postController.js
exports.getPosts = async (req, res) => {
  try {
    const posts = await Post.find().populate('author', 'name');
    res.json(posts);
  } catch (err) {
```

```

        res.status(500).json({ message: err.message });
    }
};

```

## 2. Sử dụng middleware để tái sử dụng logic:

- Tạo middleware để xử lý xác thực, logging, hoặc validation:

javascript

```

const validatePost = (req, res, next) => {
    if (!req.body.title || !req.body.content) {
        return res.status(400).json({ message: 'Title and content are required' });
    }
    next();
};
app.post('/posts', validatePost, authMiddleware, async (req, res) => { ... });

```

## 3. Tối ưu hóa truy vấn:

- Sử dụng lean() để trả về JSON thô nếu không cần các phương thức Mongoose:

javascript

```

Post.find().lean().exec();

```

- Kết hợp populate và select để giảm dữ liệu trả về:

javascript

```

Post.find().populate('author', 'name').select('title content').exec();

```

## 4. Logging và giám sát:

- Sử dụng thư viện như morgan để ghi log các yêu cầu HTTP:

javascript

```

const morgan = require('morgan');
app.use(morgan('dev'));

```

- Theo dõi hiệu suất API bằng công cụ như New Relic hoặc Prometheus.

## 5. Test API:

- Sử dụng Postman hoặc Jest để kiểm tra các endpoint:

javascript

```

const request = require('supertest');
const app = require('./app');

describe('POST /posts', () => {
    it('should create a new post', async () => {
        const res = await request(app)
            .post('/posts')
            .send({ title: 'Test', content: 'Content', authorId: '123' });
        expect(res.statusCode).toEqual(201);
        expect(res.body).toHaveProperty('title', 'Test');
    });
});

```

## Tình huống tổng hợp thực tế

Giả sử bạn đang xây dựng một ứng dụng blog:

- **Populate:** Khi hiển thị bài viết, bạn sử dụng populate để lấy thông tin tác giả và bình luận:

javascript

```

app.get('/posts/:id', async (req, res) => {
  try {
    const post = await Post.findById(req.params.id)
      .populate('author', 'name avatar')
      .populate({
        path: 'comments',
        populate: { path: 'user', select: 'name avatar' }
      })
      .lean();
    if (!post) return res.status(404).json({ message: 'Post not found' });
    res.json(post);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

- **Regex:** Tạo một endpoint tìm kiếm bài viết theo từ khóa:  
javascript

```

app.get('/search', async (req, res) => {
  try {
    const keyword = escapeRegex(req.query.q);
    const posts = await Post.find({
      $or: [
        { title: { $regex: keyword, $options: 'i' } },
        { content: { $regex: keyword, $options: 'i' } }
      ]
    }).populate('author', 'name').lean();
    res.json(posts);
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
});

```

- **REST API:** Xây dựng các endpoint CRUD với xác thực:  
javascript

```

app.post('/posts', authMiddleware, validatePost, async (req, res) => {
  try {
    const post = new Post({
      title: req.body.title,
      content: req.body.content,
      author: req.user.id
    });
    await post.save();
    res.status(201).json(post);
  } catch (err) {
    res.status(400).json({ message: err.message });
  }
});

```

## Kết luận

- **Populate:** Tối ưu hóa truy vấn liên quan, nhưng cần cẩn thận với hiệu suất và dữ liệu null.
- **Regex:** Hữu ích cho tìm kiếm linh hoạt, nhưng cần index và sanitize input để đảm bảo hiệu suất và bảo mật.
- **REST API:** Yêu cầu thiết kế tốt về cấu trúc, bảo mật, và xử lý lỗi để tạo ra API mạnh mẽ, dễ mở rộng.

## EXERCISE

### " Blog API "

#### 🔧 Dự án: Blog API với Users- Posts- Comments

##### 🔗 Tóm tắt mục tiêu

Xây dựng một hệ thống Blog gồm người dùng (Users), bài viết (Posts), và bình luận (Comments), có đầy đủ chức năng CRUD thông qua REST API. Dự án giúp bạn luyện tập:

- Thiết kế mô hình dữ liệu liên kết (relation-like)
- Sử dụng `populate` để lấy thông tin liên kết
- Tìm kiếm bằng regex
- Quản lý cấu trúc mã theo mô-đun
- Kiểm thử bằng Postman

##### 💡 Chức năng chính

- Đăng ký người dùng
- Tạo bài viết
- Bình luận bài viết
- Lấy danh sách bài viết kèm comment và thông tin người dùng
- Tìm kiếm người dùng theo tên/email (regex)
- Thao tác CRUD đầy đủ với tất cả các thực thể

##### 📁 Công nghệ sử dụng

| Công nghệ                  | Mục đích                                |
|----------------------------|-----------------------------------------|
| Node.js                    | Server backend                          |
| Express.js                 | Framework cho REST API                  |
| MongoDB Compass            | Quản lý cơ sở dữ liệu MongoDB trực quan |
| MongoDB Atlas (hoặc local) | Cơ sở dữ liệu                           |
| Mongoose                   | ORM thao tác với MongoDB                |
| Postman                    | Kiểm thử REST API                       |
| VS Code                    | IDE                                     |
| dotenv                     | Quản lý biến môi trường                 |
| Windows                    | Hệ điều hành phát triển                 |

##### 📁 Cấu trúc thư mục

pgsql

blog-api/

```
├── models/
│   ├── User.js
│   ├── Post.js
│   └── Comment.js
├── routes/
│   ├── users.js
│   ├── posts.js
│   └── comments.js
├── controllers/
│   ├── userController.js
│   ├── postController.js
│   └── commentController.js
```

```

|
|— config/
|   |— db.js
|
|— app.js
|— .env
|— package.json

```

### Tóm tắt các bước thực hiện

1. Cài đặt môi trường
2. Khởi tạo dự án Node.js
3. Tạo kết nối MongoDB với Compass
4. Tạo models (User, Post, Comment)
5. Thiết lập routes và controllers
6. Cấu hình `populate` và tìm kiếm bằng regex
7. Kiểm thử API bằng Postman
8. Hoàn thiện và kiểm tra

### Chi tiết từng bước

#### **Bước 1: Cài đặt môi trường**

- Cài Node.js: <https://nodejs.org>
- Cài MongoDB Compass: <https://www.mongodb.com/try/download/compass>
- Cài VS Code: <https://code.visualstudio.com>

#### **Bước 2: Khởi tạo dự án**

bash

```

mkdir blog-api
cd blog-api
npm init -y
npm install express mongoose dotenv
npm install --save-dev nodemon

```

Tạo file `.env`:

`env`

```

PORT=5000
MONGODB_URI=mongodb://127.0.0.1:27017/blogdb

```

Tạo file `app.js`:

`js`

```

require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');

const app = express();
app.use(express.json());

const userRoutes = require('./routes/users');
const postRoutes = require('./routes/posts');
const commentRoutes = require('./routes/comments');

app.use('/api/users', userRoutes);
app.use('/api/posts', postRoutes);
app.use('/api/comments', commentRoutes);

mongoose.connect(process.env.MONGODB_URI)

```

```

.then(() => {
  app.listen(process.env.PORT, () =>
    console.log(`Server running on port ${process.env.PORT}`)
  );
})
.catch(err => console.log(err));

```

### **Bước 3: Tạo models**

**models/User.js**

js

```

const mongoose = require('mongoose');

const userSchema = new mongoose.Schema({
  username: String,
  email: String
});

module.exports = mongoose.model('User', userSchema);

```

**models/Post.js**

js

```

const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  title: String,
  content: String,
  author: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  comments: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Comment' }]
});

module.exports = mongoose.model('Post', postSchema);

```

**models/Comment.js**

js

```

const mongoose = require('mongoose');

const commentSchema = new mongoose.Schema({
  text: String,
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  post: { type: mongoose.Schema.Types.ObjectId, ref: 'Post' }
});

module.exports = mongoose.model('Comment', commentSchema);

```

### **Bước 4: Tạo controllers**

**controllers/userController.js**

js

```

const User = require('../models/User');

exports.createUser = async (req, res) => {
  const user = await User.create(req.body);
  res.status(201).json(user);
};

exports.searchUser = async (req, res) => {
  const q = req.query.q;
  const users = await User.find({
    $or: [
      { username: { $regex: q, $options: 'i' } },
      { email: { $regex: q, $options: 'i' } }
    ]
  });
};

```



```

    ]
  });
  res.json(users);
};
controllers/postController.js
js

const Post = require('../models/Post');

exports.createPost = async (req, res) => {
  const post = await Post.create(req.body);
  res.status(201).json(post);
};

exports.getPostWithComments = async (req, res) => {
  const post = await Post.findById(req.params.id)
    .populate('author')
    .populate({
      path: 'comments',
      populate: { path: 'user' }
    });
  res.json(post);
};
controllers/commentController.js
js

const Comment = require('../models/Comment');
const Post = require('../models/Post');

exports.createComment = async (req, res) => {
  const comment = await Comment.create(req.body);
  await Post.findByIdAndUpdate(req.body.post, {
    $push: { comments: comment._id }
  });
  res.status(201).json(comment);
};

```



## **Bước 5: Tạo routes**

**routes/users.js**

js

```

const express = require('express');
const router = express.Router();
const { createUser, searchUser } = require('../controllers/userController');

```

```

router.post('/', createUser);
router.get('/search', searchUser);

```

```

module.exports = router;

```

**routes/posts.js**

js

```

const express = require('express');
const router = express.Router();
const { createPost, getPostWithComments } =
  require('../controllers/postController');

```

```

router.post('/', createPost);
router.get('/:id', getPostWithComments);

```

```

module.exports = router;

```

**routes/comments.js**

js

```
const express = require('express');
const router = express.Router();
const { createComment } = require('../controllers/commentController');

router.post('/', createComment);

module.exports = router;
```

## ✅ Bước 6: Dùng Postman kiểm thử

Import các request:

1. POST /api/users – Tạo user  
json

```
{
  "username": "alice",
  "email": "alice@example.com"
}
```

2. POST /api/posts – Tạo post  
json

```
{
  "title": "First Post",
  "content": "This is a blog post",
  "author": "<user_id>"
}
```

3. POST /api/comments – Tạo comment  
json

```
{
  "text": "Nice article!",
  "user": "<user_id>",
  "post": "<post_id>"
}
```

4. GET /api/posts/<post\_id> – Xem bài viết kèm comment và user

5. GET /api/users/search?q=alice – Tìm user với regex

## ✅ Tổng kết

Bạn đã học được cách:

- Cấu trúc một project Node.js chuyên nghiệp
- Sử dụng Mongoose để thiết lập liên kết giữa các bảng
- Dùng `populate()` để hiển thị dữ liệu quan hệ
- Tìm kiếm bằng regex trong MongoDB
- Quản lý RESTful API hoàn chỉnh với Postman

## PHÂN TÍCH QUY LUẬT THỰC THI

PROMT: Tôi là người mới học, tôi cần phát hiện quy luật thực thi của chương trình. Hãy Mô tả quá trình thực thi của code của bài tập

Chúng ta sẽ mô tả quá trình thực thi của chương trình theo **thứ tự dòng chảy**, từ lúc chạy `node server.js` đến khi người dùng gọi API như `GET /api/tasks`.

## LUYỆN TẬP

### Bài Tập 1: Populate Comments with User Info



#### Mục tiêu:

- Hiểu và sử dụng `populate()` trong Mongoose để tự động lấy thông tin liên quan từ collection khác.



#### Chức năng:

- API lấy danh sách bài viết và hiển thị kèm thông tin người comment (username, email).



#### Công nghệ sử dụng:

- Node.js, Express, MongoDB, Mongoose



#### Cấu trúc thư mục:

pgsql

```
project/
├── models/
│   ├── Post.js
│   ├── User.js
│   └── Comment.js
├── routes/
│   └── posts.js
├── app.js
└── .env
```



#### Các bước thực hiện:

1. Tạo models User, Post, Comment
2. Mỗi comment chứa ref đến user
3. Trong API GET `/posts/:id`, sử dụng `populate("comments.user")`
4. Trả về JSON gồm thông tin bài viết và các comment đã populate



#### Test Cases:

- GET `/posts/:id`: Kiểm tra xem các comment có chứa thông tin user không.
- Nếu comment không có user, response phải bỏ qua hoặc hiển thị null.

### Bài Tập 2: Match với MongoDB Regex



#### Mục tiêu:

- Sử dụng biểu thức chính quy để tìm kiếm dữ liệu linh hoạt trong MongoDB.



#### Chức năng:

- API tìm kiếm người dùng theo tên hoặc email với từ khóa không phân biệt chữ hoa/thường.



#### Công nghệ sử dụng:

- Node.js, Express, MongoDB, Mongoose

## Cấu trúc thư mục:

bash

```
project/
├── models/
│   └── User.js
├── routes/
│   └── users.js
├── app.js
└── .env
```

## Các bước thực hiện:

1. Tạo model User với các field username, email
2. Tạo route GET /users/search?q=keyword
3. Dùng biểu thức chính quy: { \$regex: query, \$options: "i" }
4. Trả về danh sách user khớp

## Test Cases:

- /users/search?q=anh: tìm user tên "Anh", "anh", "ANH"
- /users/search?q=@gmail: tìm user có email @gmail

## Bài Tập 3: Xây dựng REST API hoàn chỉnh

### Mục tiêu:

- Xây dựng một RESTful API server với đầy đủ chức năng CRUD cho một blog system.

### Chức năng:

- Tạo, đọc, cập nhật, xóa bài viết
- Comment vào bài viết
- Tự động populate user trong comment

### Công nghệ sử dụng:

- Node.js, Express, MongoDB, Mongoose
- Postman để test API

## Cấu trúc thư mục:

pgsql

```
project/
├── models/
│   ├── User.js
│   ├── Post.js
│   └── Comment.js
├── routes/
│   ├── posts.js
│   ├── users.js
│   └── comments.js
├── controllers/
│   ├── postController.js
│   ├── userController.js
│   └── commentController.js
├── app.js
└── .env
```

## Các bước thực hiện:

1. Cài đặt Express + Mongoose
2. Tạo models: User, Post, Comment
3. Tạo route CRUD cho từng tài nguyên
4. Dùng populate để lấy thông tin user trong comment

## 5. Viết middleware xử lý lỗi & validate



### Test Cases:

- POST /users: tạo user mới
- POST /posts: tạo bài viết mới
- POST /comments: comment bài viết (gắn user)
- GET /posts/:id: đọc bài viết với comments đã populate user
- PUT /posts/:id: cập nhật nội dung bài viết
- DELETE /posts/:id: xóa bài viết

## TỪ ĐIỂN THUẬT NGỮ (FOUNDATION GLOSSARY)

### 1. Nhóm thuật ngữ về Mongoose & Quan hệ dữ liệu (Populate)

| Thuật ngữ              | Ý nghĩa kỹ thuật                     | Giải thích thực tế                                                                                                                 |
|------------------------|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>Populate</b>        | Liên kết dữ liệu (Linking Documents) | Quá trình thay thế các <code>ObjectId</code> bằng nội dung chi tiết của bản ghi tương ứng từ collection khác.                      |
| <b>Reference (Ref)</b> | Tham chiếu                           | Việc lưu trữ ID của một bản ghi này trong một bản ghi khác để tạo mối liên hệ (giống Foreign Key trong SQL).                       |
| <b>Path</b>            | Đường dẫn trường dữ liệu             | Tên của trường (field) trong Schema mà bạn muốn thực hiện lệnh populate.                                                           |
| <b>Model</b>           | Đối tượng mô phỏng                   | Một class cung cấp giao diện để tương tác với các collection trong database (CRUD).                                                |
| <b>Schema</b>          | Lược đồ dữ liệu                      | Cấu trúc định nghĩa các trường thông tin, kiểu dữ liệu và các ràng buộc (validation) cho một Document.                             |
| <b>N+1 Problem</b>     | Vấn đề truy vấn N+1                  | Một lỗi về hiệu năng khi code thực hiện 1 query chính để lấy danh sách, sau đó lại chạy thêm N query phụ để lấy dữ liệu liên quan. |

### 2. Nhóm thuật ngữ về Tìm kiếm (Regex & Query)

| Thuật ngữ                         | Ý nghĩa kỹ thuật           | Giải thích thực tế                                                                                                       |
|-----------------------------------|----------------------------|--------------------------------------------------------------------------------------------------------------------------|
| <b>Regex (Regular Expression)</b> | Biểu thức chính quy        | Một chuỗi ký tự đặc biệt dùng làm "mẫu" để tìm kiếm hoặc thao tác trên các chuỗi văn bản.                                |
| <b>Pattern</b>                    | Mẫu tìm kiếm               | Cấu trúc cụ thể trong Regex để máy tính đối soát (ví dụ: <code>/^abc/</code> là tìm chuỗi bắt đầu bằng abc).             |
| <b>Case-insensitive</b>           | Không phân biệt hoa thường | Tùy chọn (flag <code>i</code> trong regex) cho phép tìm kiếm "Admin" và "admin" là như nhau.                             |
| <b>Wildcard</b>                   | Ký tự đại diện             | Các ký tự như <code>.</code> <code>*</code> trong regex đại diện cho bất kỳ ký tự nào, dùng để tìm kiếm một phần của từ. |
| <b>Indexing</b>                   | Đánh chỉ mục               | Việc tạo ra một "mục lục" cho database giúp việc tìm kiếm dữ liệu diễn ra tức thì thay vì phải quét toàn bộ bảng.        |
| <b>Collation</b>                  | Quy tắc đối soát           | Bộ quy tắc xác định cách so sánh và sắp xếp các ký tự (rất quan trọng khi xử lý tiếng Việt có dấu).                      |

### 3. Nhóm thuật ngữ về Kiến trúc REST API (Express & Mongoose)

| Thuật ngữ           | Ý nghĩa kỹ thuật             | Giải thích thực tế                                                                                             |
|---------------------|------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>REST API</b>     | Giao diện lập trình ứng dụng | Một tiêu chuẩn thiết kế API dựa trên các phương thức HTTP (GET, POST, PUT, DELETE) để quản lý tài nguyên.      |
| <b>Middleware</b>   | Phần mềm trung gian          | Các hàm nằm giữa Request và Response, dùng để kiểm tra đăng nhập, log dữ liệu hoặc xử lý lỗi.                  |
| <b>Controller</b>   | Tầng điều khiển              | Nơi tiếp nhận request từ client, điều phối dữ liệu từ Model và trả về kết quả (Response).                      |
| <b>Status Code</b>  | Mã trạng thái HTTP           | Các con số tiêu chuẩn trả về để báo kết quả (200: OK, 201: Created, 404: Not Found, 500: Server Error).        |
| <b>Payload</b>      | Gói dữ liệu                  | Phần dữ liệu thực tế được gửi đi trong thân (Body) của một HTTP Request hoặc Response.                         |
| <b>Sanitization</b> | Làm sạch dữ liệu             | Quá trình loại bỏ các ký tự nguy hiểm khỏi input của người dùng để chống tấn công (như NoSQL Injection).       |
| <b>Stateless</b>    | Không lưu trạng thái         | Đặc điểm của REST: Mỗi request phải chứa đầy đủ thông tin để server hiểu, server không "nhớ" request trước đó. |

#### Chủ đề 1: Quan hệ dữ liệu & Populate (Mongoose)

Mục tiêu: Hiểu sâu về hiệu năng và thiết kế cơ sở dữ liệu (Schema Design).

- **Prompt 1 (Hiệu năng Populate vs Aggregation):**

"Hãy đóng vai một chuyên gia MongoDB. Hãy phân tích và so sánh hiệu năng giữa việc sử dụng Mongoose Populate và MongoDB Aggregation Pipeline (`$lookup`) khi xử lý dữ liệu lớn (ví dụ: 1 triệu bản ghi comments). Khi nào tôi nên từ bỏ Populate để chuyển sang Aggregation để tối ưu hóa REST API?"

- **Prompt 2 (Thiết kế Schema: Embedding vs Referencing):**

"Trong bối cảnh xây dựng tính năng Comment cho một bài viết trên blog có lưu lượng truy cập cao. Hãy đánh giá ưu và nhược điểm (Trade-offs) giữa hai chiến lược thiết kế Schema: 'Embedding' (nhúng comment vào post) và 'Referencing' (lưu comment riêng và dùng ID để liên kết). Chiến lược nào tối ưu hơn cho tốc độ đọc (Read) và chiến lược nào tốt hơn cho tốc độ ghi (Write)?"

- **Prompt 3 (Vấn đề N+1 trong NoSQL):**

"Hãy giải thích khái niệm 'N+1 problem' thường gặp trong SQL. Liệu vấn đề này có tồn tại khi sử dụng Mongoose populate trong Node.js không? Hãy phân tích cách Mongoose thực thi populate dưới background và đề xuất cách xử lý nếu tôi cần populate danh sách user cho 1000 comments cùng lúc."

#### Chủ đề 2: Tìm kiếm & Regex (MongoDB)

Mục tiêu: Hiểu về Indexing, Performance và Bảo mật trong tìm kiếm.

- **Prompt 4 (Hiệu năng Regex & Indexing):**

"Hãy phân tích tác động của việc sử dụng Regex (`$regex`) lên hiệu năng của CPU và RAM server MongoDB khi tìm kiếm trên một field không có Index (COLLSCAN) so với field có Index. Tại sao tìm kiếm regex bắt đầu bằng ký tự wildcards (ví dụ: `/.*keyword/`) lại không thể tận dụng Index hiệu quả?"

- **Prompt 5 (Text Search vs Regex):**

"Hãy so sánh tính năng Text Index (`$text search`) của MongoDB và Regex thông thường. Trong trường hợp nào tôi nên sử dụng Text Index thay vì Regex để xây dựng tính năng tìm kiếm cho một trang thương mại điện tử? Đánh giá về độ chính xác và tốc độ."



- **Prompt 6 (Bảo mật - ReDoS Attack):**

"Hãy đánh giá rủi ro bảo mật liên quan đến ReDoS (Regular Expression Denial of Service) khi cho phép user nhập trực tiếp chuỗi tìm kiếm vào query Regex của MongoDB trong Node.js. Hãy đề xuất các giải pháp hoặc thư viện (như `escape-string-regexp`) để ngăn chặn lỗ hổng này."

### **Chủ đề 3: Kiến trúc REST API (Express & Mongoose)**

Mục tiêu: Tổ chức code, Xử lý lỗi và Tiêu chuẩn hóa API.

- **Prompt 7 (Mô hình MVC & Layered Architecture):**

"Hãy phân tích lợi ích của việc tách biệt logic trong Express App thành các tầng: Controller, Service và Repository (Data Access Layer). Tại sao việc viết tất cả logic truy vấn Mongoose ngay trong Controller (như bài lab cơ bản) lại gây khó khăn cho việc Unit Test và bảo trì (Maintainability) khi dự án mở rộng?"

- **Prompt 8 (Pagination Strategies):**

"Khi xây dựng API `GET /comments` với hàng triệu bản ghi, hãy đánh giá ưu nhược điểm của hai kỹ thuật phân trang: 'Offset-based Pagination' (dùng `skip` và `limit`) và 'Cursor-based Pagination' (dùng `_id` hoặc `timestamp`). Tại sao Offset-based lại bị chậm dần khi số lượng trang tăng lên?"

- **Prompt 9 (Validation & Data Integrity):**

"So sánh việc Validate dữ liệu ở cấp độ Mongoose Schema (built-in validation) và việc sử dụng một thư viện validation riêng biệt ở tầng Middleware (như Joi hoặc Zod). Tại sao các dự án lớn thường ưu tiên validate ngay tại Request body trước khi dữ liệu chạm tới tầng Model?"

- **Prompt 10 (Error Handling Best Practices):**

"Hãy đánh giá tầm quan trọng của việc xây dựng một 'Centralized Error Handling Middleware' trong Express.js. Làm thế nào để chuẩn hóa responses lỗi (Error responses) trả về cho Client (frontend) thay vì để lộ stack trace hoặc lỗi thô từ MongoDB driver?"

## 10 PROMPT TỰ NGHIÊN CỨU THỰC TIỄN CHUYÊN NGHIỆP

### I. Tối ưu hóa hiệu năng Mongoose & Database (Advanced Performance)

Trong các dự án lớn, việc query chậm vài mili-giây nhân với hàng triệu request sẽ là thảm họa.

- **Prompt 1 (The Power of `.lean()`):**

"Trong Mongoose, hãy giải thích cơ chế hoạt động của hàm `.lean()`. Tại sao sử dụng `.lean()` lại giúp tăng hiệu năng query đáng kể đối với các thao tác GET (Read-only)? Hãy so sánh memory footprint (lượng RAM tiêu thụ) của một Mongoose Document đầy đủ so với một Plain Old JavaScript Object (POJO) trả về từ `.lean()`."

- **Prompt 2 (Virtual Populate vs Physical References):**

"Hãy so sánh kỹ thuật `Virtual Populate` trong Mongoose với việc lưu trữ mảng `References` cứng (`Array of ObjectIds`). Trong trường hợp thiết kế database cho hệ thống có quan hệ 1-Nhiều cực lớn (ví dụ: 1 User có 1 triệu Log activities), giải pháp nào giúp tránh việc document vượt quá giới hạn 16MB của MongoDB và tối ưu hóa việc query?"

- **Prompt 3 (Database Sharding & Populate):**

"Khi hệ thống MongoDB mở rộng quy mô bằng kỹ thuật `Sharding`, tính năng `$lookup` (Aggregation) và `Populate` của Mongoose gặp những hạn chế kỹ thuật nào khi dữ liệu nằm rải rác trên các Shard khác nhau? Hãy đề xuất chiến lược thiết kế Schema để giảm thiểu việc join dữ liệu giữa các Shard."

- **Prompt 4 (Caching Strategy for Heavy Queries):**

"Hãy thiết kế một chiến lược `Caching` (sử dụng Redis) cho một API Endpoint có sử dụng `populate` phức tạp và `regex search`. Làm thế nào để implement pattern 'Cache-Aside' trong Node.js để giảm tải cho database? Cách xử lý `Cache Invalidation` (xóa cache) khi dữ liệu gốc thay đổi để đảm bảo tính nhất quán?"

- **Prompt 5 (Mongoose Transactions - ACID):**

"Mặc dù MongoDB là NoSQL, nhưng từ v4.0 đã hỗ trợ `Multi-document Transactions`. Hãy tạo ví dụ code Node.js sử dụng `Mongoose Session` để đảm bảo tính toàn vẹn dữ liệu (ACID) khi thực hiện xóa một User và đồng thời xóa tất cả Comments của user đó. Điều gì xảy ra nếu server crash giữa chừng?"

### II. Tìm kiếm nâng cao & Xử lý ngôn ngữ (Advanced Search)

Regex chỉ là bước khởi đầu. Thực tế đòi hỏi tìm kiếm thông minh hơn và nhanh hơn.

- **Prompt 6 (Vietnamese Diacritics & Collation):**

"Làm thế nào để cấu hình MongoDB và Mongoose để thực hiện tìm kiếm tiếng Việt không phân biệt dấu (Accent-insensitive search)? Ví dụ: tìm từ khóa 'hà nội' nhưng vẫn

ra kết quả chứa 'Hà Nội'. Hãy giải thích về Collation trong MongoDB và cách áp dụng nó vào query."

- **Prompt 7 (Atlas Search vs Regex):**

"So sánh hiệu năng và tính năng giữa việc dùng \$regex truyền thống và MongoDB Atlas Search (dựa trên Apache Lucene). Tại sao các dự án lớn lại chuyển sang Atlas Search để làm các tính năng như Autocomplete, Fuzzy matching (tìm gần đúng) thay vì dùng Regex?"

- **Prompt 8 (Fuzzy Search Implementation):**

"Nếu không dùng Text Index, làm thế nào để implement thuật toán tìm kiếm mờ (Fuzzy Search) bằng Regex trong Node.js để xử lý lỗi chính tả của người dùng? (Ví dụ: user gõ 'ipone' vẫn tìm ra 'iphone'). Phân tích độ phức tạp và rủi ro về hiệu năng."

- **Prompt 9 (Compound Indexes & Query Optimization):**

"Hãy giải thích quy tắc ESR (Equality, Sort, Range) khi tạo Index trong MongoDB. Nếu tôi có một API tìm kiếm User vừa filter theo role (chính xác), vừa search theo name (regex), vừa sort theo createdAt, tôi cần tạo Compound Index như thế nào để query đạt tốc độ tối đa?"

### III. Kiến trúc REST API chuẩn Enterprise (Architecture & Security)

Viết API chạy được là dễ, viết API dễ sống sót qua đợt tấn công hoặc tải cao mới khó.

- **Prompt 10 (Dependency Injection & Testability):**

"Trong Express.js, làm thế nào để áp dụng nguyên lý Dependency Injection (DI) thủ công hoặc dùng thư viện (như InversifyJS/Tsyringe) để tách biệt Service và Model? Tại sao việc này lại quan trọng cốt yếu để viết Unit Test (sử dụng Jest/Sinon) mà không cần kết nối tới Database thật?"

- **Prompt 11 (API Rate Limiting & Throttling):**

"Hãy hướng dẫn cách bảo vệ Public API khỏi các cuộc tấn công DDoS hoặc Brute-force bằng cách sử dụng Middleware express-rate-limit kết hợp với Redis Store. Sự khác biệt giữa Rate Limiting (giới hạn số request) và Throttling (làm chậm tốc độ xử lý) là gì?"

- **Prompt 12 (API Versioning Strategies):**

"Hãy phân tích các chiến lược đánh phiên bản cho REST API (API Versioning): URL versioning (/v1/users), Header versioning (Accept-Version), và Query Param versioning. Ưu nhược điểm của từng loại và cách tổ chức thư mục project Node.js để maintain nhiều phiên bản API cùng lúc?"

- **Prompt 13 (Graceful Shutdown):**

"Trong môi trường Production (Kubernetes/Docker), làm thế nào để implement 'Graceful Shutdown' cho Node.js server? Cần viết code như thế nào để đảm bảo khi server nhận tín hiệu SIGTERM, nó sẽ hoàn thành nốt các request đang xử lý dở, đóng kết nối Mongoose an toàn rồi mới tắt hẳn?"

- **Prompt 14 (Automated Documentation - OpenAPI/Swagger):**

"Thay vì viết document thủ công, hãy giới thiệu quy trình 'Code-first' để tự động sinh ra Swagger UI (OpenAPI 3.0) từ code Express/Mongoose. Làm thế nào để comment trong code hoặc dùng thư viện (như `swagger-jsdoc`) để document luôn đồng bộ với code thực tế?"

- **Prompt 15 (Security Headers & Sanitization):**

"Ngoài xác thực (AuthN/AuthZ), hãy liệt kê các lớp bảo mật cần thiết cho một REST API Node.js. Giải thích vai trò của `Helmet` (secure HTTP headers), `HPP` (Http Parameter Pollution prevention), và `MongoSanitize` (chống NoSQL Injection). Tại sao chỉ validate dữ liệu đầu vào là chưa đủ?"